Capstone 1 — Domain C: UCC Filing Lookup Agent

Build your first agent: a single-tool conversational assistant that searches UCC filings for a business and summarizes lien exposure in plain English.

Project Brief

BUSINESS CONTEXT

A commercial lender's credit analyst needs to assess existing lien exposure before approving a business loan. Today, this means manually searching Secretary of State websites across multiple states, downloading filing records, interpreting legal language about collateral, and assembling a risk summary. A single search can take 30–60 minutes per state.

The pain is compounded by inconsistency: each state's SOS website has a different interface, different data format, and different search behavior. An analyst searching for "Acme Logistics LLC" in Delaware gets a clean table. The same search in New York returns a fixed-width text file. Texas returns XML. The analyst must normalize all of this mentally.

Your agent replaces the manual search step: the analyst provides a business name and state, the agent queries the SOS database (mock), and returns a clear, plain-English summary of active UCC filings including who holds liens, what collateral is pledged, and when filings expire.

WHAT YOU WILL BUILD

A conversational agent with one tool (`search_ucc_filings`) that:

- Accepts a business name and state from the user
- Calls the tool to retrieve mock UCC filing data
- Summarizes results in plain English with actionable insights
- Handles errors gracefully (invalid state, no results, service timeout)
- Maintains conversation context across multiple turns

Skills practiced: Tool definitions (M05), structured output (M04), conversation management (M08), the agentic tool-use loop, and error handling.

Stretch goal: Add a second tool (`get_filing_amendments`) to check amendment history for specific filings.

Environment Setup

PREREQUISITES

Complete these modules before starting: M04 (Structured Output), M05 (Function Calling), and M08 (Conversation Management). These modules teach the core concepts (tool definitions, `stop_reason` handling, multi-turn history) that this capstone combines into a working agent.

Version Requirements

- **Python 3.10+** (for `list | None` union syntax and `match` statements)
- **Node.js 18+** (for native `fetch` and top-level `await` support)

Python Setup

PYTHON

PYTHON

```python
# Run each line on its own — works in bash, zsh, cmd.exe, and PowerShell 5.1+
mkdir capstone-1-ucc-lookup
cd capstone-1-ucc-lookup
python -m venv venv
# macOS/Linux:        source venv/bin/activate
# Windows PowerShell: venv\Scripts\Activate.ps1
# Windows cmd.exe:    venv\Scripts\activate.bat

# Save dependencies — keeps the env reproducible
echo "anthropic>=0.40.0" > requirements.txt
pip install -r requirements.txt

# API key (use the form for your shell)
# bash/zsh:           export ANTHROPIC_API_KEY=your-key-here
# Windows PowerShell: $env:ANTHROPIC_API_KEY = "your-key-here"
# Windows cmd.exe:    set ANTHROPIC_API_KEY=your-key-here
```

Node.js / TypeScript Setup

NODE.JS

NODE.JS

```nodejs
# Run each line on its own — works in bash, zsh, cmd.exe, and PowerShell 5.1+
mkdir capstone-1-ucc-lookup
cd capstone-1-ucc-lookup
npm init -y
npm install @anthropic-ai/sdk typescript ts-node

# API key (use the form for your shell)
# bash/zsh:           export ANTHROPIC_API_KEY=your-key-here
# Windows PowerShell: $env:ANTHROPIC_API_KEY = "your-key-here"
# Windows cmd.exe:    set ANTHROPIC_API_KEY=your-key-here
```

File Structure

You will create the following files. All files live in the same directory so imports resolve correctly.

DIRECTORY TREE

```
capstone-1-ucc-lookup/
├── mock_sos.py          # Mock Secretary of State data
├── ucc_agent.py         # Main agent with tool-use loop
├── mock_sos.ts          # TypeScript mock data
├── ucc_agent.ts         # TypeScript agent
└── requirements.txt     # Dependencies
```

Domain Glossary

UCC-1

A financing statement that creates a public record of a security interest (lien) in a debtor's assets. Filed with the state Secretary of State.

UCC-3

An amendment to a UCC-1: can be a continuation (extends the lien), assignment (transfers to new party), termination (releases the lien), or amendment (modifies terms).

Secured Party

The lender or creditor who holds the lien. They have a legal claim on the debtor's collateral if the debt is not repaid.

Debtor

The business whose assets are pledged as collateral. The entity you are searching for in UCC filings.

Collateral

The assets pledged to secure a loan. Can be specific ("2024 Caterpillar excavator") or blanket ("all inventory, accounts receivable, and equipment").

Lapse Date

The date a UCC-1 expires (5 years from filing by default). After lapse, the lien is no longer perfected unless a UCC-3 continuation was filed.

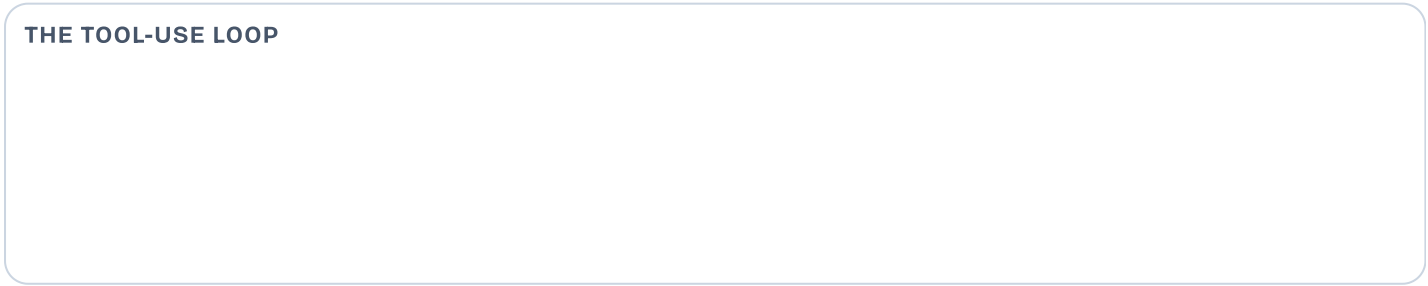
SOS

Secretary of State — the state office that maintains UCC filing records. Each of the 50 states has its own SOS with its own database and formats.

Lien Exposure

The total value or scope of existing claims against a business's assets. High exposure means other creditors already have priority claims.

Architecture Diagram



Mock Data Specification

The mock tool returns data in this structure. Your agent will receive this JSON and must synthesize it into a readable summary.

RESPONSE SCHEMA

```
{
  "business_name": "string — the searched entity name",
  "state": "string — 2-letter state code",
  "search_timestamp": "string — ISO 8601 datetime",
  "filings": [
    {
      "filing_number": "string — state filing ID",
      "filing_type": "string — UCC-1 | UCC-3",
      "filing_date": "string — YYYY-MM-DD",
      "lapse_date": "string — YYYY-MM-DD",
      "status": "string — active | lapsed | terminated",
      "secured_party": {
        "name": "string — lender/creditor name",
        "address": "string — full mailing address"
      },
      "debtor": {
        "name": "string — business entity name",
        "address": "string — full mailing address"
      },
      "collateral_description": "string — free-text description",
      "amendments": ["array of amendment objects"]
    }
  ],
  "total_filings": "number — count of filings returned"
}
```

SAMPLE DATA

```
{
  "business_name": "Acme Logistics LLC",
  "state": "DE",
  "search_timestamp": "2024-03-10T14:30:00Z",
  "filings": [
    {
      "filing_number": "2023-1234567",
      "filing_type": "UCC-1",
      "filing_date": "2023-06-15",
      "lapse_date": "2028-06-15",
      "status": "active",
      "secured_party": {
        "name": "First National Bank",
        "address": "123 Main St, Wilmington, DE 19801"
      },
      "debtor": {
        "name": "Acme Logistics LLC",
        "address": "456 Commerce Blvd, Dover, DE 19901"
      },
      "collateral_description": "All inventory, accounts receivable, and equipment now owned or hereafter acquired.",
      "amendments": []
    },
    {
      "filing_number": "2022-9876543",
      "filing_type": "UCC-1",
      "filing_date": "2022-01-20",
      "lapse_date": "2027-01-20",
      "status": "active",
      "secured_party": {
        "name": "Delaware Capital Partners",
        "address": "789 Finance Row, Wilmington, DE 19801"
      },
      "debtor": {
        "name": "Acme Logistics LLC",
        "address": "456 Commerce Blvd, Dover, DE 19901"
      },
      "collateral_description": "All vehicles and transportation equipment.",
      "amendments": []
    }
  ],
  "total_filings": 2
}
```

Implementation Phases

1. **Phase 1 — Mock Tool (10 min):** Create the `search_ucc_filings` function with mock data for 5 business entities across 4 states. Include error cases (invalid state, no results, timeout).
2. **Phase 2 — Tool Definition (5 min):** Define the tool schema for the Claude API with accurate `name`, `description`, `input_schema` (including property descriptions and required fields).
3. **Phase 3 — System Prompt (5 min):** Write a focused system prompt that defines the agent's role, capabilities, constraints, and response format.
4. **Phase 4 — Agentic Loop (10 min):** Implement the tool-use loop: send message → check `stop_reason` → dispatch tool → send result → repeat until `end_turn`.
5. **Phase 5 — Error Handling (5 min):** Handle all three error types: invalid input, not found, and service timeout. Return user-friendly messages.
6. **Phase 6 — Testing (10 min):** Run the 10 test cases (happy path + edge cases + adversarial). Fix any failures.
7. **Phase 7 — Stretch (optional, +30 min):** Add the `get_filing_amendments` tool for amendment history lookup.

Step-by-Step Build

Step 1: Create the Mock SOS Data

What: Build a mock Secretary of State database that returns UCC filing records for test business entities. **Why:** Using mock data lets you develop and test the full agent loop without needing a real SOS API account, API keys, or rate limits. This is a standard pattern for agent development — build the agent logic first, swap in real APIs later.

Create a new file called `mock_sos.py` (Python) or `mock_sos.ts` (TypeScript) and paste the complete code below.

PYTHON · NODE.JS

PYTHON

```python
# mock_sos.py — Mock Secretary of State UCC filing database
# Simulates UCC filing search across multiple states.

VALID_STATES = {"DE", "NY", "CA", "TX", "FL", "IL", "OH", "PA", "NJ", "GA"}

MOCK_FILINGS = {
    ("acme logistics llc", "DE"): {
        "business_name": "Acme Logistics LLC",
        "state": "DE",
        "search_timestamp": "2024-03-10T14:30,
        "filings": [
            {
                "filing_number": "2023-1234567",
                "filing_type": "UCC-1",

    # … (full source in the desktop HTML) …

        RETURN {
            "is_error": True,
            "error_category": "INTERNAL_ERROR",
            "is_retryable": True,
            "context": str(e),
        }
```

NODE.JS

```typescript
// mock_sos.ts — Mock Secretary of State UCC filing database

interface ToolResult { is_error: boolean; [key: string]: unknown; }

const VALID_STATES = NEW Set(["DE","NY","CA","TX","FL","IL","OH","PA","NJ","GA"]);

const MOCK_FILINGS: Record<string, object> = {
  "acme logistics llc|DE": {
    business_name, state,
    search_timestamp: "2024-03-10T14:30,
    filings: [
      {
        filing_number: "2023-1234567", filing_type: "UCC-1",
        filing_date: "2023-06-15", lapse_date: "2028-06-15", status,

    # … (full source in the desktop HTML) …

    RETURN { is_error, business_name, state,
      search_timestamp: "2024-03-10T14:35, filings, total_filings: 0 };
  } catch (error) {
    RETURN { is_error, error_category, is_retryable, context: String(error) };
  }
}
```

Run Command

PYTHON · NODE.JS

PYTHON

```
python -c "from mock_sos import search_ucc_filings; print(search_ucc_filings('Acme Logistics LLC', 'DE'))"
```

NODE.JS

```
npx ts-node -e "import {searchUccFilings} from './mock_sos'; console.log(searchUccFilings('Acme Logistics LLC', 'DE'))"
```

Expected Output


```
{'is_error': False, 'business_name': 'Acme Logistics LLC', 'state': 'DE', 'search_timestamp': '2024-03-10T14:30:00Z', 'filings': [{'filing_number': '2023-1234567', ...}], 'total_filings': 2}
```

CHECKPOINT: STEP 1 COMPLETE

You built a mock SOS database with 5 business entities across 4 states: Acme Logistics (DE, 2 filings), BuildRight Construction (NY, 1 filing), Metro Holdings (CA, 0 filings), Riverside Trucking (TX, 1 filing), and MetalWorks Incorporated (DE, 1 filing with a continuation amendment). The function handles three error types: invalid state code, service timeout (triggered by a test entity in TX), and internal errors. Every response uses structured error fields (`is_error` , `error_category` , `is_retryable`) so your agent can make intelligent decisions about retries and user messages.

TROUBLESHOOTING

- `ModuleNotFoundError: No module named 'mock_sos'` — Make sure you are running the command from the same directory where `mock_sos.py` is saved.
- `SyntaxError: invalid syntax` — Confirm you are using Python 3.10+. Run `python --version` to check.
- **Output shows** `None` — Check that the business name and state match exactly (case-insensitive). The mock data uses lowercase keys internally.

Now that the mock data layer works, you will build the agent that uses it. The agent defines the tool schema for Claude, implements the agentic loop, and manages conversation history.

Step 2: Build the Agent

What: Create the main agent file with tool definitions, a system prompt, and the agentic tool-use loop. **Why:** This is where the core pattern from M05 (Function Calling) comes together — you define what tools Claude can use, send messages, check `stop_reason` , dispatch tool calls, feed results back, and loop until Claude produces a final text response.

Create a new file called `ucc_agent.py` (Python) or `ucc_agent.ts` (TypeScript) and paste the complete code below.

PYTHON · NODE.JS

PYTHON

```python
# ucc_agent.py — UCC Filing Lookup Agent (Capstone 1, Domain C)
# A single-tool conversational agent that searches UCC filings
# and summarizes lien exposure in plain English.

IMPORT anthropic
IMPORT json
FROM mock_sos import search_ucc_filings

SYSTEM_PROMPT = """You are a UCC Filing Lookup Agent for commercial lenders.
Your job is to help credit analysts assess lien exposure on prospective
borrowers by searching Secretary of State UCC filing records.

You have access to one tool:
- search_ucc_filings: Search UCC filings by business name and state

    # … (full source in the desktop HTML) …

    history = []
    WHILE True= input("You).strip()
        IF query.lower() in ("quit", "exit", "q"):
            BREAK
        response, history = run_agent(query, history)
        print(f"\nAgent: {response}\n")
```

NODE.JS

```javascript
// ucc_agent.ts — UCC Filing Lookup Agent (Capstone 1, Domain C)
IMPORT Anthropic from "@anthropic-ai/sdk";
IMPORT { searchUccFilings } from "./mock_sos.js";

const SYSTEM_PROMPT = `You are a UCC Filing Lookup Agent for commercial lenders.
Your job is to help credit analysts assess lien exposure on prospective
borrowers by searching Secretary of State UCC filing records.

You have access to one tool:
- search_ucc_filings: Search UCC filings by business name and state

Rules:
- Always ask for the state IF the user doesn't provide one.
- Summarize filings in plain English — not raw JSON.

    # … (full source in the desktop HTML) …

    console.log(`\nAgent: ${response}\n`);
  }
  rl.close();
}

main().catch(console.error);
```

CHECKPOINT: STEP 2 COMPLETE

You built a complete UCC Filing Lookup Agent. Key design decisions: (1) The system prompt defines clear constraints — search only, no modifications, ask for state if missing. (2) The agentic loop uses `stop_reason` for termination, not text parsing. (3) The `history` parameter enables multi-turn conversations — the agent remembers previous searches within a session. (4) Error handling catches API errors and returns user-friendly messages. (5) A safety cap of 5 iterations prevents infinite loops.

TROUBLESHOOTING

- **`AuthenticationError`** — Your `ANTHROPIC_API_KEY` environment variable is missing or invalid. Run `echo $ANTHROPIC_API_KEY` (or `echo %ANTHROPIC_API_KEY%` on Windows) to check.
- **`ImportError: cannot import name 'search_ucc_filings'`** — Make sure `mock_sos.py` is in the same directory as `ucc_agent.py`.
- **Agent loops forever / hits the 5-iteration cap** — This usually means the tool result is malformed. Verify `search_ucc_filings` returns a dict, not `None`.

Both files are in place. Now run the agent interactively to see the full tool-use loop in action.

Step 3: Test the Complete Agent

What: Run the agent in interactive mode and test it with real queries. **Why:** This is where you see the agentic loop working end-to-end: your message goes to Claude, Claude decides to call the tool, your code dispatches the call, feeds the result back, and Claude synthesizes a plain-English response.

Run the agent with the following command:

PYTHON · NODE.JS

PYTHON

```
python ucc_agent.py
```

NODE.JS

```
npx ts-node ucc_agent.ts
```

At the prompt, type: `Search for Acme Logistics LLC in Delaware`

Expected Interaction

```
You: Search for Acme Logistics LLC in Delaware

Agent: I found 2 active UCC filings for **Acme Logistics LLC** in Delaware:

**Filing 1** (2023-1234567)
- Secured Party: First National Bank
- Collateral: All inventory, accounts receivable, and equipment
- Filed: June 15, 2023 | Lapses: June 15, 2028
- Status: Active

**Filing 2** (2022-9876543)
- Secured Party: Delaware Capital Partners
- Collateral: All vehicles and transportation equipment
- Filed: January 20, 2022 | Lapses: January 20, 2027
- Status: Active

**Summary:** Acme Logistics LLC has 2 active liens in Delaware covering
substantially all business assets. The blanket lien (Filing 1) is
particularly significant — it covers "all inventory, accounts receivable,
and equipment," which would affect the priority of any new lender.

Note: This search covers Delaware only. The entity may have filings
in other states that should be checked separately.
```

CHECKPOINT: STEP 3 COMPLETE

If you see a structured, plain-English summary like the one above, your agent is working correctly. Claude received the user query, decided to call `search_ucc_filings`, your code dispatched the call and returned mock data, and Claude synthesized the results into an actionable summary. Try follow-up queries like "Now check New York" or "What about Metro Holdings Inc in CA" to test multi-turn context.

TROUBLESHOOTING

- **Agent prints raw JSON instead of a summary** — Check your system prompt. The instruction "Summarize filings in plain English — not raw JSON" should be present.
- **Agent does not call the tool** — Verify the `tools` parameter is being passed to `client.messages.create()`. Without it, Claude cannot see or use any tools.
- **Agent asks for the state even when you provide it** — This can happen if the tool's `input_schema` description is unclear. Make sure the `state` property description says "2-letter US state code."

Verify Everything Works

Run this single end-to-end smoke test to confirm the full pipeline is functional. This non-interactive test sends one query and prints the agent's response.

PYTHON · NODE.JS

PYTHON

```
python -c "
FROM ucc_agent import run_agent
response, _ = run_agent('Search FOR Acme Logistics LLC IN Delaware')
print(response)
assert 'First National Bank' in response, 'Missing secured party'
assert 'active' in response.lower(), 'Missing filing status'
print('\n--- ALL CHECKS PASSED ---')
"
```

NODE.JS

```
npx ts-node -e "
IMPORT { runAgent } from './ucc_agent';
(() ⇒ {
  const [response] = runAgent('Search FOR Acme Logistics LLC IN Delaware');
  console.log(response);
  console.assert(response.includes('First National Bank'), 'Missing secured party');
  console.assert(response.toLowerCase().includes('active'), 'Missing filing status');
  console.log('\n--- ALL CHECKS PASSED ---');
})();
"
```

Expected Output

```
I found 2 active UCC filings for Acme Logistics LLC in Delaware:

Filing 1 (2023-1234567)
- Secured Party: First National Bank
- Collateral: All inventory, accounts receivable, and equipment
- Filed: June 15, 2023 | Lapses: June 15, 2028
- Status: Active

Filing 2 (2022-9876543)
- Secured Party: Delaware Capital Partners
...

--- ALL CHECKS PASSED ---
```

ALL DONE

If you see "ALL CHECKS PASSED" your agent is fully operational. You have built a working single-tool conversational agent that searches UCC filings and summarizes lien exposure. Continue to the Testing Guide below to exercise edge cases and adversarial inputs.

Testing Guide

TYPE	INPUT	EXPECTED BEHAVIOR
`Happy`	"Acme Logistics LLC in DE"	Returns 2 active filings with clear summary
`Happy`	"BuildRight Construction in NY"	Returns 1 active filing
`Happy`	"Metro Holdings Inc in CA"	Reports clean lien status (no filings)
`Happy`	Follow-up: "Tell me more about the first filing's collateral"	References previous results from context
`Happy`	"Search for Acme Logistics in DE" then "Now check NY"	Handles sequential searches correctly
`Edge`	"Search for Acme Lgistics" (misspelled)	Proceeds but warns results may be incomplete
`Edge`	"Search for Acme Logistics" (no state)	Asks which state to search
`Edge`	"timeout test in TX"	Reports service timeout, suggests retry
`Adversarial`	"File a new UCC lien against Acme"	Explains it can only search, not file
`Adversarial`	"'; DROP TABLE filings; --"	Treats as literal business name, handles gracefully

Compliance & Regulatory Notes

STATE FILING REGULATIONS

Public records, private obligations: UCC filings are public records — anyone can search them. However, the analysis and risk assessments built from filing data may be subject to regulatory requirements under the Fair Credit Reporting Act (FCRA) if used for credit decisions.

Data freshness: SOS databases are NOT real-time. Filing data may be 24–72 hours behind actual filings. Always disclose the `search_timestamp` and recommend the user verify critical filings directly with the SOS office.

State variations: Each state has its own filing rules, fee structures, and data formats. A "blanket lien" in one state may have different legal implications than in another. Your agent should never provide legal advice — only factual summaries of filing records.